

No Under 40 — Codebase Security & Performance Review

Date: April 19, 2026
Project: No Under 40 (Dance School Platform)
Scope: Full codebase review for security vulnerabilities and performance bottlenecks
Reviewers: Claude Code Analysis

EXECUTIVE SUMMARY

Comprehensive analysis identified **24 refactoring opportunities** across the codebase:

- **5 CRITICAL** issues (security/financial impact)
- **6 HIGH** priority issues (data integrity/performance)
- **7 MEDIUM** severity issues (audit/reliability)
- **6 LOW** severity issues (code quality/UX)

Immediate Action Required: Fix 5 critical items before next production deployment.

CRITICAL ISSUES (Severity: CRITICAL)

1. Unrestricted School Admin PATCH Endpoint — Mass Assignment Vulnerability

File: `src/app/api/hq/schools/[id]/route.ts` (line 22)
Type: Security — Mass Assignment / Privilege Escalation
Severity: CRITICAL

Description:

The PATCH handler accepts `await admin.from('schools').update(body).eq('id', id)` directly without field-level filtering. An HQ user could inject arbitrary fields like:

- `platform_fee_percentage` (financial impact)
- `stripe_account_id` (payment hijacking)
- `active` (school lockout)
- `user_id` (ownership change)

The endpoint only checks HQ role but not specific permissions (e.g., Finance role cannot modify platform fees, but this code allows it).

Impact:

🚨 HQ team members could modify school configurations, bypass platform fees, activate/deactivate schools they shouldn't, or hijack Stripe accounts.

Suggested Approach:

Implement field whitelist: only allow updates to specific fields (name, address, phone, logo_url, etc.). Cross-check HQ sub-role against permission matrix in `src/lib/hq-permissions.ts` before allowing sensitive fields.

Fix Priority: Do immediately

2. SQL Injection Risk in Student Lessons Query — Course Location Fields

File: `src/app/api/student/lessons/route.ts` (lines 38-50)
Type: Security — Potential Injection
Severity: CRITICAL

Description:

The endpoint queries `courses.country` and `courses.city`, but the database schema shows courses are FK-linked to schools, NOT locations. The code manually compares `course.country` and `course.city` which don't exist in the schema, suggesting these fields may be user-controlled or incorrectly mapped.

The string comparison logic:

```
cc.includes(fc.slice(0, 3))
```

on user input is unsafe and can be bypassed or exploited.

Impact:

🚨 Logical error or data corruption. If these fields are populated from user input elsewhere, string matching could be bypassed or exploited. Students may see incorrect lessons or no results.

Suggested Approach:

Verify schema — `courses` should reference `school_rooms` → `school_locations` for location data. Query via proper FK joins. Validate user input (city/country) against a whitelist of known values before matching.

Fix Priority: Do immediately

3. Missing Rate Limiting on Auth Endpoints

Files:

- `src/app/api/auth/register/route.ts`
- `src/app/api/auth/send-reset/route.ts`

Type: Security — Account Enumeration / Brute Force
Severity: CRITICAL

Description:

No rate limiting on password reset or registration endpoints. Attackers can:

1. Enumerate valid emails via timing attacks (reset endpoint will succeed for existing emails, fail for non-existent ones)
2. Brute-force weak passwords on registration
3. Perform mass account creation attacks

Impact:

🚨 Account enumeration, credential stuffing, mass registration attacks, potential DoS.

Suggested Approach:

Add rate limiting middleware (IP-based, 5-10 requests/minute) on `/api/auth/send-reset` and `/api/auth/register`. Use library like `@upstash/ratelimit` or implement in Next.js middleware.

Fix Priority: Do immediately

4. Attendance Marking Logic Inverted — Credit Burn Status Reversed

File: `src/app/api/attendance/[lessonId]/route.ts` (lines 142, 152-161)
Type: Logic Error / Financial Impact
Severity: CRITICAL

Description:

The attendance status mapping is backwards:

```
status: statusMap[r.status_id]?.burns_credit ? 'present' : 'no_show',
```

If `burns_credit` is true (e.g., status is "No-show" with `burns_credit=true`), it sets status to 'present'. This inverts the logic:

- ✗ No-shows are marked as present
- ✗ Credits are NOT burned when they should be
- ✗ Cancellation policy is not enforced

Impact:

🔴 **FINANCIAL CRITICAL** — Students not attending lessons have their credits refunded. Schools lose revenue. The entire cancellation policy is not enforced. This is a direct revenue loss.

Suggested Approach:

Fix the logic:

```
status: statusMap[r.status_id]?.burns_credit ? 'no_show' : 'present'
```

Or use explicit mapping without ternary confusion.

Fix Priority: 🟡 Do IMMEDIATELY (revenue impact)

5. Stripe Webhook Account Verification Missing

File: `src/app/api/webhooks/stripe/route.ts` (entire handler)

Type: Security — Webhook Spoofing / CSRF

Severity: 🟡 CRITICAL

Description:

The webhook handler does NOT verify the Stripe account ID (`event.account`) against the metadata. An attacker could send a webhook with another school's `school_id` in metadata, and the handler would process credits for the wrong student.

While Stripe signature IS verified, account isolation is missing. This is a **multi-tenant isolation break**.

Impact:

🔴 **MULTI-TENANT BREACH** — One school can trigger credit additions for students at another school. Attacker can:

- Issue refunds across schools
- Activate subscriptions for random students
- Manipulate student balances across the entire network

Suggested Approach:

After signature verification, validate that `event.account` matches the school's `stripe_account_id` in the metadata. Extract `school_id` from metadata and cross-check it against the Stripe account's stored school relationship.

Fix Priority: 🟡 Do immediately (multi-tenant security)

HIGH-SEVERITY ISSUES

6. Missing Permissions Check on HQ PATCH School Endpoint

File: `src/app/api/hq/schools/[id]/route.ts`

Type: Security — Privilege Escalation

Severity: 🟡 HIGH

Description:

Only checks for HQ role, but does not enforce sub-role permissions. A "Support" or "Analytics" HQ member can call this endpoint and modify school data they shouldn't access:

- Finance role should control platform fees → no check
- Operations role should manage schools → no check
- Support role should not modify anything → no check

Impact:

Unauthorized role escalation. Support staff could modify school platform fees or activate/deactivate schools.

Suggested Approach:

Implement permission check

```
hasPermission(profile.hq_sub_role, 'schools_create_edit')
```

Cross-check with `src/lib/hq-permissions.ts` for each field being updated.

Fix Priority: 🟡 High (next sprint)

7. N+1 Query on School Students Endpoint

File: `src/app/api/school/students/route.ts` (lines 43-83)

Type: Performance — N+1 Query

Severity: 🟡 HIGH

Description:

The endpoint fetches data in separate queries:

1. `school_students` (separate query)
2. All `students` (separate query) ✓
3. `student_packages` (separate query) ✓
4. `student_subscriptions` (separate query) — But doesn't include `student_id`, requiring broken client-side mapping
5. Loops through data and maps packages/subscriptions O(n)

For 1000 students: **3+ database roundtrips instead of 1 joined query**.

Impact:

Wrong subscriptions shown for students. Slow API responses at scale (10K+ bookings per school).

Suggested Approach:

Batch join with proper FK mapping:

```
.select('school_students(*), students(*), student_packages(*), student_subscriptions(*)')
```

Or use explicit joins with `student_id` in subscription select.

Fix Priority: 🟡 High (performance at scale)

8. Metadata Type Mismatch in Stripe Checkout

File: `src/app/api/stripe/checkout/route.ts` (lines 139-158)
Type: Security / Logic Error
Severity:  HIGH

Description:
The code stores `student_id: user.id` (`auth.users.id`) in Stripe metadata, but the database expects `student_id` to be the UUID from the `students` table. Two different ID types:

- Metadata stores: `auth.users.id` (Supabase auth user ID)
- Database expects: `students.id` (students table ID — different!)
- Webhook uses: metadata `student_id` directly to insert into `student_packages` **✗ Wrong type**

Impact:
Student packages inserted with wrong `student_id` (auth ID instead of students table ID). Bookings and credit queries fail or return wrong data. Students won't receive credits after paying.

Suggested Approach:
Store `student.id` (not `user.id`) in metadata. Or fetch `students.id` and validate before inserting webhook data. Ensure type consistency throughout.

Fix Priority:  High (payment processing)

9. Calendar iCal Feed — No Pagination or Limit

File: `src/app/api/calendar/[schoolId]/route.ts` (lines 24-39)
Type: Performance / Denial of Service
Severity:  HIGH

Description:
The iCal endpoint queries all future lessons (`gte('date', today)`) **without pagination or a date limit**. A school with thousands of annual lessons will generate:

- Massive iCal file (100KB+)
- Slow exports
- External calendar app timeouts
- Potential DOS if many users subscribe

Impact:
Slow external calendar syncs. Large bandwidth usage. External calendar apps (Google Calendar, Apple Calendar) may timeout or fail.

Suggested Approach:

- Limit to next 6 months: `.lte('date', futureDate)`
- Paginate or offer date-range filters
- Implement caching with Supabase CDN
- Add compression (gzip)

Fix Priority:  High (external integration reliability)

10. Missing Discount Code Validation in Subscriptions

File: `src/app/api/stripe/checkout/route.ts` (lines 168-226)
Type: Security — Lost Revenue
Severity:  HIGH

Description:
Discount codes are only validated for "package" type (lines 86-104), but **NOT for subscriptions** (lines 168-226). Subscription checkout skips all discount validation, allowing:

- Students to bypass discount codes entirely
- Attackers to inject discounts server-side
- Revenue loss from missing discount logic

Impact:
Lost revenue from missing discount logic on subscriptions. Unfair pricing for different customers. Subscription customers don't get their promised discounts.

Suggested Approach:
Apply same discount validation logic to subscriptions. Extract shared validation into a function to avoid duplication and ensure parity.

Fix Priority:  High (revenue impact)

11. Missing Input Validation on Booking Lesson Search

File: `src/app/api/student/lessons/route.ts` (lines 9-15)
Type: Security — Potential Injection or Abuse
Severity:  HIGH

Description:
Query parameters (`city`, `country`, `language`, `from`, `to`) are used directly in Supabase queries with minimal sanitization:

```
// Line 39: No length limit, could cause regex explosion
courseQuery.ilike('city', `${city}%`)

// Line 48: Dangerous substring matching
cc.includes(fc.slice(0, 3))

// Line 59: No enum validation (should be IT/EN/FR/ES)
eq('courses.language', language)

// Line 29: No date format validation
gte('date', from)
```

Impact:

- Regex DOS on `ilike` queries
- Query injection if input not properly encoded
- Logical errors from malformed dates
- Performance degradation from unconstrained queries

Suggested Approach:
Validate inputs before use:

- Date format: ISO 8601 (`YYYY-MM-DD`)
- Language: enum validation (IT/EN/FR/ES)
- City: max 100 chars, no special chars
- Country: whitelist against HQ locations
- Use parameterized queries (Supabase handles this, but validate types)

Fix Priority:  High (query stability)

MEDIUM-SEVERITY ISSUES

12. No Foreign Key Constraints on Bookings

File: Database schema (migrations)
Type: Database Design / Data Integrity
Severity: ☐ MEDIUM

Description:
The bookings table has no FK constraints on:

- student_package_id
- student_subscription_id

This allows orphaned records if a package/subscription is deleted. The attendance API manually joins bookings with profiles using student_id, which is error-prone.

Impact:
Data inconsistency. Orphaned booking records can cause queries to fail or return incomplete data.

Suggested Approach:
Add FK constraints

```
REFERENCES student_packages(id) ON DELETE CASCADE
REFERENCES student_subscriptions(id) ON DELETE CASCADE
```

Fix Priority: ☐ Medium (next sprint)

13. Insufficient Logging on Admin Operations

Files: All src/app/api files
Type: Security — Audit Trail Gap
Severity: ☐ MEDIUM

Description:
Admin operations have minimal logging:

- School creation
- Team member removal
- Compensation edits
- Payment refunds

No timestamp, no IP address, no "changed from X to Y" audit trails.

Impact:
Unable to audit who made critical changes. Compliance risk (GDPR, financial regulations). Fraud detection impossible.

Suggested Approach:
Create an audit_logs table and log all admin mutations with:

- user_id
- action (created/updated/deleted)
- resource (school/teacher/discount_code/etc)
- old_value
- new_value
- timestamp
- ip_address

Fix Priority: ☐ Medium (compliance)

14. Missing Database Indexes on Foreign Keys

Files: Database schema
Type: Performance — Missing Indexes
Severity: ☐ MEDIUM

Description:
Tables like bookings, student_packages, attendance have FK columns without indexes. Queries filter by student_id, school_id, lesson_id frequently, but no indexes exist (only 2 strategic indexes in entire migration).

Impact:
Full table scans on common queries. Slow API responses at scale (10K+ bookings per school).

Suggested Approach:
Add indexes on:

```
CREATE INDEX idx_bookings_student_id ON bookings(student_id);
CREATE INDEX idx_bookings_lesson_id ON bookings(lesson_id);
CREATE INDEX idx_bookings_school_id ON bookings(school_id);
CREATE INDEX idx_student_packages_student_id ON student_packages(student_id);
CREATE INDEX idx_student_packages_school_id ON student_packages(school_id);
CREATE INDEX idx_attendance_lesson_id ON attendance(lesson_id);
CREATE INDEX idx_transactions_school_id ON transactions(school_id);
CREATE INDEX idx_messages_conversation_id ON messages(conversation_id);
```

Fix Priority: ☐ Medium (query performance)

15. Incomplete Error Handling in Stripe Webhook

File: src/app/api/webhooks/stripe/route.ts (entire)
Type: Reliability — Error Recovery
Severity: ☐ MEDIUM

Description:
Webhook handler silently ignores errors

- Package not found → logged but no retry/alert
- RPC call to increment_discount_usage could fail silently
- No DLQ (dead letter queue) or retry mechanism
- If a webhook fails mid-processing, credits may not be added but payment is taken

Impact:
Payment processing failures go unnoticed. Students don't receive credits after paying. Refund requests spike.

Suggested Approach:
Implement error recovery:

- Log all errors to Sentry
- Implement exponential backoff retries on DB errors
- Create a webhook event queue to re-process failures
- Webhook status should be stored and monitored

Fix Priority: ☐ Medium (reliability)

16. XSS Risk in Calendar iCal Description

File: `src/app/api/calendar/[schoolId]/route.ts`

Type: Security — XSS (Indirect)

Severity: ☐ MEDIUM

Description:

Lesson data is embedded directly into iCal text without escaping. iCal clients parse the DESCRIPTION field; if lesson names or teacher names contain special characters (colon, newline, semicolon), the iCal format breaks.

Example:

```
Lesson name: "Flex: 1:1 Coaching" -> iCal breaks on colon
```

Impact:

iCal feeds malformed. External calendar apps fail to import. Potential for injection if lesson name comes from user input.

Suggested Approach:

Escape iCal special characters in DESCRIPTION fields. Use iCal library (e.g., `ical.js`) to generate valid iCal instead of string concatenation.

Fix Priority: ☐ Medium (external reliability)

17. No CORS/CSRF Protection

Files: All API routes

Type: Security — CSRF / CORS

Severity: ☐ MEDIUM

Description:

No explicit CORS headers or CSRF token validation. Next.js API routes are vulnerable to cross-origin requests if a malicious site embeds a form or fetch call.

Impact:

CSRF attacks possible. Attacker can trigger bookings, refunds, or team member deletions from external sites.

Suggested Approach:

- Implement CSRF tokens on state-changing endpoints (POST/PUT/DELETE)
- Add CORS headers restricting to same-origin only (or whitelisted domains)
- Use SameSite cookie policy

Fix Priority: ☐ Medium (web security)

18. Missing Webhook Retry & Idempotency Keys

File: `src/app/api/webhooks/stripe/route.ts`

Type: Reliability — Duplicate Processing

Severity: ☐ MEDIUM

Description:

Stripe will retry webhooks if it doesn't receive a 2xx response. The handler processes every webhook naively, so a retry could process the same payment twice:

- ❌ Duplicate credits added
- ❌ Subscriptions activated twice
- ❌ Double-billing

Impact:

Double-billing. Students receive 2x credits for 1 payment. Refund requests and support burden spike.

Suggested Approach:

Store Stripe event IDs in a `processed_webhook_events` table to detect duplicates:

```
CREATE TABLE processed_webhook_events (  
  id TEXT PRIMARY KEY,  
  processed_at TIMESTAMP DEFAULT NOW()  
);
```

Check before processing, return 200 immediately after idempotency check.

Fix Priority: ☐ Medium (payment reliability)

19. Unbounded Limit on Booking History Query

File: `src/app/api/bookings/route.ts` (line 36)

Type: Performance — DoS

Severity: ☐ MEDIUM

Description:

The GET request returns **ALL bookings** for a student (`.order('booked_at')`) without pagination or limit. A student who booked 10K lessons will get a massive JSON response.

Impact:

Slow page load. Memory bloat. Network timeout. Mobile users experience significant lag.

Suggested Approach:

Add pagination:

```
?limit=50&offset=0 // Cursor-based pagination
```

Or default to last 3 months of bookings.

Fix Priority: ☐ Medium (UX at scale)

20. Weak Password Policy Enforcement

Files: Client-side (not in scope) + auth flow

Type: Security — Weak Credentials

Severity: ☐ MEDIUM

Description:
Supabase Auth is used for authentication, but there's no visible password policy enforcement:

- No minimum length
- No complexity requirements (uppercase, number, symbol)
- No admin password reset function
- Teachers and school staff may set weak passwords

Impact:
Credential stuffing attacks succeed. Weak account takeovers possible.

Suggested Approach:
Enforce password policy in Supabase auth settings:

- Min 12 chars
- Complexity: uppercase, number, symbol
- Send security notifications on password changes

Fix Priority: ☐ Medium (account security)

LOW-SEVERITY ISSUES

21. Unused Imports and Dead Code

Files: Multiple API routes
Type: Code Quality
Severity: ☒ LOW

Description:
ESLint disabled on some imports; unused variables in query results.

Suggested Approach:
Enable ESLint rules, remove unused imports. Use TypeScript strict mode.

Fix Priority: ☒ Low (tech debt)

22. Missing Timezone Handling

Files: Calendar, booking, attendance
Type: Logic — Timezone Issues
Severity: ☒ LOW

Description:
Dates are stored in UTC (ISO 8601), but lesson start times are TIME (no timezone). A 3pm lesson in Rome (UTC+1) will show as 3pm UTC globally. If a student in Tokyo books, they're booking based on wrong time.

Impact:
Timezone confusion. Students book wrong times.

Suggested Approach:
Store lesson times with timezone (e.g., `lesson_timezone: Europe/Rome`). Convert times on display based on user's timezone preference.

Fix Priority: ☒ Low (future enhancement)

23. Missing Bulk Operations Endpoints

Files: School students, packages, etc.
Type: Performance / UX
Severity: ☒ LOW

Description:
No bulk delete, bulk update endpoints. Schools must delete `students/packages` one at a time, causing N API calls.

Suggested Approach:
Add batch endpoints:

```
POST /api/school/students/bulk-delete
PATCH /api/school/packages/bulk-update
```

Fix Priority: ☒ Low (UX improvement)

24. No Soft Deletes

Files: Database schema
Type: Data Integrity
Severity: ☒ LOW

Description:
Tables use hard DELETE, not soft deletes (`deleted_at` flag). Historical data is lost. Cannot easily restore deleted records.

Impact:
Data loss. Compliance issue for audit trails.

Suggested Approach:
Add `deleted_at` `TIMESTAMPZ` column to critical tables. Filter out deleted rows in RLS policies.

Fix Priority: ☒ Low (data compliance)

SUMMARY TABLE

#	Issue	File(s)	Severity	Type	Impact
1	Unrestricted School Admin PATCH	/hq/schools/[id]/route.ts	☒ CRITICAL	Security	Multi-tenant + Financial
2	SQL Injection in Lessons Query	/student/lessons/route.ts	☒ CRITICAL	Security	Data Corruption
3	Missing Rate Limiting on Auth	/auth/register, /send-reset	☒ CRITICAL	Security	Account Enumeration
4	Inverted Attendance Logic	/attendance/[lessonId]/route.ts	☒ CRITICAL	Logic Error	Financial (Revenue Loss)
5	Missing Stripe Account Verification	/webhooks/stripe/route.ts	☒ CRITICAL	Security	Multi-Tenant Breach
6	Missing HQ Permissions Check	/hq/schools/[id]/route.ts	☒ HIGH	Security	Privilege Escalation
7	N+1 Query on School Students	/school/students/route.ts	☒ HIGH	Performance	Slow API (1000+ students)
8	Metadata Type Mismatch in Stripe	/stripe/checkout/route.ts	☒ HIGH	Logic Error	Payment Processing
9	No Pagination on iCal Feed	/calendar/[schoolId]/route.ts	☒ HIGH	Performance	External Integration
10	Missing Discount Validation on Subscriptions	/stripe/checkout/route.ts	☒ HIGH	Security	Revenue Loss

11 Missing Input Validation on Lessons Search	/student/lessons/route.ts	🔴 HIGH	Security	Query Injection DOS
12 Missing FK Constraints	Database schema	🟡 MEDIUM	Data Integrity	Orphaned Records
13 Insufficient Admin Logging	All /api	🟡 MEDIUM	Security	Audit Trail Gap
14 Missing Database Indexes	Database schema	🟡 MEDIUM	Performance	Full Table Scans
15 Incomplete Error Handling in Webhook	/webhooks/stripe/route.ts	🟡 MEDIUM	Reliability	Undetected Failures
16 XSS Risk in iCal Descriptions	/calendar/[schoolId]/route.ts	🟡 MEDIUM	Security	Format Corruption
17 Missing CORS/CSRF Protection	All API routes	🟡 MEDIUM	Security	CSRF Attacks
18 Missing Webhook Idempotency	/webhooks/stripe/route.ts	🟡 MEDIUM	Reliability	Double-Billing
19 Unbounded Booking History Query	/bookings/route.ts	🟡 MEDIUM	Performance	Slow Page Load
20 Weak Password Policy	Supabase Auth config	🟡 MEDIUM	Security	Weak Credentials
21 Unused Imports	Multiple	🟢 LOW	Quality	Code Bloat
22 Missing Timezone Handling	Calendar, booking	🟢 LOW	Logic	UX Issue
23 No Bulk Operations	Admin endpoints	🟢 LOW	UX	Admin Friction
24 No Soft Deletes	Database schema	🟢 LOW	Data Integrity	Data Recovery Gap

IMMEDIATE ACTION ITEMS (Next Sprint)

Do These First (Before Next Production Deployment)

1. Fix Attendance Logic (Issue #4) 🔥
 - File: `src/app/api/attendance/[lessonId]/route.ts`, line 142
 - Change: Invert the status mapping `burns_credit ? 'no_show' : 'present'`
 - Reason: Revenue loss — no-shows aren't burned currently
 - Estimated Time: 15 minutes
 - Testing: Verify no-show marks credit as burned in test
2. Add Stripe Account Verification (Issue #5) 🔥
 - File: `src/app/api/webhooks/stripe/route.ts`
 - Change: Cross-check `event.account` against school `stripe_account_id`
 - Reason: Multi-tenant security breach
 - Estimated Time: 20 minutes
 - Testing: Attempt webhook from wrong school account (should fail)
3. Fix Metadata Type Mismatch (Issue #8) 🔥
 - File: `src/app/api/stripe/checkout/route.ts` & webhook handler
 - Change: Store `students.id` not `auth.users.id` in metadata
 - Reason: Payment processing broken for subscriptions
 - Estimated Time: 30 minutes
 - Testing: Verify `student_id` matches in stripe metadata vs DB
4. Add Field Whitelist to School PATCH (Issue #1) 🔥
 - File: `src/app/api/hq/schools/[id]/route.ts`
 - Change: Only allow updates to specific fields
 - Reason: Prevent mass assignment attacks
 - Estimated Time: 20 minutes
 - Testing: Attempt to update `platform_fee` (should fail)
5. Add Rate Limiting to Auth Endpoints (Issue #3) 🔥
 - Files: `/api/auth/register`, `/api/auth/send-reset`
 - Change: IP-based rate limiter (5-10 req/min)
 - Reason: Prevent account enumeration and brute force
 - Estimated Time: 45 minutes (includes @upstash/ratelimit setup)
 - Testing: Verify 429 response after 10 attempts

NEXT PRIORITIES (Following Sprint)

6. Add Permissions Check to HQ School PATCH (Issue #6)
7. Fix N+1 Query on School Students (Issue #7)
8. Add iCal Pagination (Issue #9)
9. Add Discount Code Validation to Subscriptions (Issue #10)
10. Input Validation on Lesson Search (Issue #11)

ESTIMATED EFFORT

Priority	Issues	Est. Time	Sprint
🔴 Critical (Fix immediately)	5	2-3 hours	This sprint
🟡 High (Next sprint)	6	8-12 hours	Next sprint
🟡 Medium (Backlog)	7	20-30 hours	Following sprints
🟢 Low (Nice to have)	6	10-15 hours	Tech debt

DEPLOYMENT CHECKLIST

Before deploying to production, ensure:

- ☐ Issue #4 fixed (attendance logic)
- ☐ Issue #5 fixed (Stripe account verification)
- ☐ Issue #8 fixed (metadata type)
- ☐ Issue #1 fixed (field whitelist)
- ☐ Issue #3 fixed (rate limiting)
- ☐ All 5 fixes tested in staging
- ☐ No regressions in existing tests
- ☐ Performance benchmarks maintained

CONCLUSION

The codebase has a solid foundation but requires **urgent attention to 5 critical issues** before production scaling. The most critical risks are:

1. **Financial** — Attendance logic inverted, discounts not applied
2. **Security** — Multi-tenant isolation broken, no rate limiting, mass assignment
3. **Performance** — Missing indexes, unbounded queries, N+1 patterns

Recommend prioritizing the 5 critical fixes this sprint, then addressing the 6 high-priority issues in the following sprint.

Report Generated: April 19, 2026
Reviewed By: Claude Code Analysis
Status: Awaiting developer action